

challenges during a security audit due to insufficient logging and monitoring of its containerised applications. The auditors discovered that the organisation had limited visibility into network traffic between containers, making it difficult to identify unauthorised access attempts or data exfiltration. As a result, the institution was required to implement extensive monitoring solutions to comply with regulatory standards.

Misconfigured containers:

Misconfiguration is a common issue that can lead to serious security vulnerabilities in containerised environments. With many settings and options available, it's easy to overlook critical security configurations. For instance, running containers as root users or enabling unnecessary capabilities can expose the application to potential attacks.

Use case: A company deploying a web application in a container forgot to set the appropriate security context for their pods in Kubernetes. As a result, the pods ran with elevated privileges, allowing attackers to execute arbitrary code within the container. This oversight led to a breach where sensitive user data was compromised, demonstrating the importance of secure default configurations.

Supply chain vulnerabilities:

The container supply chain includes everything from base images to third-party libraries and dependencies. Each component introduces potential vulnerabilities, making it crucial to maintain a secure supply chain. An attack on a popular base image or a widely used library can have widespread consequences, affecting multiple organisations that rely on those components.

Real-life scenario: In 2021, researchers discovered a vulnerability in a widely used Node.js library that was commonly included in Docker images. This vulnerability allowed attackers to execute arbitrary code within applications that relied on the library.

Many organisations using these images faced immediate risks, prompting them to scramble for patches and updates to mitigate the vulnerabilities. This incident underscored the need for thorough supply chain security practices, including regular vulnerability scanning and dependency management.

Docker security best practices

Image security

One of the foundational steps in securing Docker containers is to focus on image security. The Docker image is essentially the blueprint for your container and using insecure or unverified images can lead to significant vulnerabilities. Here are some best practices for image security.

Use trusted base images: Start by using official images from Docker Hub or other reputable sources. Avoid using generic images that may contain malware or vulnerabilities.

Regularly scan images for vulnerabilities: Use tools like Trivy, Clair, or Aqua Security to scan images for known vulnerabilities before deploying them. This proactive measure can help identify and mitigate risks early in the development process.

Implement image signing and verification: Docker Content Trust (DCT) allows you to sign images and verify their authenticity before deployment. This ensures that only trusted images are used in your deployments.

Container configuration

Proper container configuration is critical for maintaining security. Here are some essential practices.

Limit container privileges: Running processes as non-root users can significantly reduce the risk of unauthorised access and privilege escalation attacks. Use the USER directive in your Dockerfile to specify a non-root user.

Configure resource limits: Setting CPU and memory limits helps prevent denial-of-service attacks and resource exhaustion. This can be done using the `--memory` and `--cpus` flags when running a container.

Use read-only file systems:

Employing read-only filesystems for containers prevents unauthorised modifications and can help protect against certain types of attacks. Use the `--read-only` flag when starting your containers.

Network security

Network security plays a vital role in protecting containerised applications. Here are key practices to enhance network security.

Implement network segmentation:

Utilise Docker networks to isolate containers from one another. This limits the exposure of sensitive services and reduces the attack surface.

Use firewalls and security groups:

Configure firewalls and security groups to restrict access to sensitive services and minimise the risk of external attacks. This includes setting up rules to allow only necessary traffic to your containers.

Secure communication between containers: Always use secure communication protocols such as TLS/SSL for data in transit between containers. This ensures that sensitive information is encrypted and protected from eavesdropping.

Logging and monitoring

Enabling comprehensive logging for container activity allows for greater visibility into operations and potential security incidents. Here are best practices for logging and monitoring.

Enable container logging: Configure logging for your containers using Docker's logging drivers or a centralised logging solution like the ELK stack (Elasticsearch, Logstash, Kibana) or Fluentd. This helps capture logs for analysis and troubleshooting.

Monitor for unusual behaviour:

Use monitoring tools to track container